⚡ Agent Mesh SRE

User Guide

Monitor · Reason · Act · Learn

Version 5.0 | July 2026 | API Days — Agent Mesh on Streaming World

---

1. Introduction

Agent Mesh SRE is a self-healing AI workflow platform that monitors Apache Kafka clusters, detects incidents, reasons about root causes, takes policy-gated actions, and learns from outcomes. It implements the M→R→A→L loop (Monitor → Reason → Act → Learn) across four specialised AI agents running against a live Confluent for Kubernetes (CFK) cluster on DigitalOcean Kubernetes Service.

1.1 Application Modes

Mode	Description
Simulator (default)	Fully in-memory simulation. No Kafka broker required. All 11 scenarios run instantly.
Real Cluster (CFK)	Connects to live Confluent for Kubernetes on DigitalOcean via KAFKA_BOOTSTRAP_SERVERS and KAFKA_BOOTSTRAP (two separate variables that must match exactly). Set KAFKA_MODE=real.
Builder	Drag-and-drop workflow editor that exports AsyncAPI 3.0 specs and Strimzi CRs.

1.2 The MRAL Loop

Phase	Agent	Action	Output
Monitor	Monitor Agent	Ingests metrics, detects anomalies	Incident raised
Reason	Monitor Agent	Root-cause analysis, confidence scoring	Reasoning logged
Act	Monitor Agent	Executes MCP tool call (gated or auto)	Action audited
Learn	Monitor Agent	Records lesson for future reference	Lesson stored

2. Dashboard Overview

Area	Purpose

Top Bar	Cluster status, mode badge (MOCK/REAL), Reset button, user avatar
Left Rail — Scenarios	Pause/Resume auto-trigger button + 11 scenario buttons. Click any to fire.
Left Rail — Kafka Topics	Live topic health, lag metrics, partition counts with healthy/unhealthy badges.
Centre Canvas	Animated agent mesh showing real-time MRAL phases and Kafka message flow.
Right Panel	Tabbed inspector: Pending Approvals, Audit Log, Notifications, Lessons Learned.
Scenario History Bar	Bottom strip — all scenario runs with 👤 Manual / 🤖 Auto badges. Click any entry to open detail popup.

2.1 Auto-Trigger Control

The top of the scenario panel shows a Pause / Resume toggle:

- ▶️ Auto-trigger active (green) — Monitor agent autonomously fires scenarios every 30–40 seconds on Vercel.
- ⏸️ Auto-trigger paused (amber) — no autonomous scenarios fire. Clicking any scenario button still triggers it immediately.
- Manually triggered scenarios are always stamped 👤 Manual regardless of pause state.

> **Tip**
> Press Escape at any time to close any open modal — scenario history popup, approval gate, lesson detail, broker info, cluster stats, or topic modal.

2.2 Scenario History Bar

Every scenario run appears as a card in the history bar. Each card shows:

- Scenario name and left-border colour (matches scenario type)
- Timestamp and confidence score
- Status badge: ✓ Resolved, ⏳ Awaiting Approval, or ✗ Rejected
- Trigger source badge: 👤 Manual (operator clicked) or 🤖 Auto (Monitor agent fired autonomously)
- Review → button on Awaiting Approval entries — opens the approval gate directly

Click any card to open the full Scenario Detail popup. For awaiting-approval entries the popup has a Send for Approval → button at the bottom to push it to the right-panel approval queue.

3. Scenario Detail Popup

Clicking any history entry opens a full incident summary. The popup contains these sections in order:

- Header — scenario name, timestamp, trigger badge (👤 Manually triggered / 🤖 Auto-triggered)
- MRAL phase badges — Monitor, Reason, Act, Learn
- 🧠 Monitor → Reason — root cause, Kafka feature cited, confidence %, rationale
- 🎯 Root Cause — red-bordered card with the single most probable cause from the Monitor agent's analysis
- ⚠️ Available Root Causes — all 4 candidate causes numbered, most probable highlighted in bold red

- ⚡ Act — action taken (or Awaiting Approval), outcome badge (✅ SUCCESS / 🚫 REJECTED), lag resolved, approved/rejected by
- 📚 Learn — lesson card with effectiveness badge, notes, adjusted threshold; or rejection note if rejected
- 📋 Live Events Timeline — chronological agent audit trail

> **Note**
> The 🎯 Root Cause and ⚠️ Available Root Causes sections appear for ALL entries — including approved, rejected, and awaiting approval — so you always have the full incident context.

4. Human-in-the-Loop Approvals

Four scenarios require human approval before the Monitor agent executes an infra-mutating MCP tool call. The approval workflow has two entry points:

- Auto-triggered approval gate — canvas pauses, a PENDING card appears in the right Approvals tab with a Review & Decide button
- History bar Review → — for any ⏳ Awaiting Approval entry, click Review → to open the gate directly
- History popup Send for Approval → — opens the entry popup, then click Send for Approval → to push to the Approvals queue

4.1 The Approval Modal

- Click Review & Decide (right panel) or Review → (history bar) to open the Policy Gate modal
- Modal shows: proposed MCP tool call and parameters, Why This Action rationale, Infrastructure mutation risk badge
- 🎯 Root Cause — most probable trigger reason (red card)
- ⚠️ Available Root Causes — all candidate causes, most likely highlighted in red
- Click Approve to proceed, Reject — No Action to abort, or ✕ / Escape to close without deciding
- The audit log records every decision with actor identity and timestamp

4.2 Scenarios Requiring Approval

Scenario	Tool Call Requiring Approval
Consumer Lag Spike	kafka.scaleConsumers — scale consumer group replicas
Share Group Rebalance	kafka.acknowledgeShareGroupRebalance — acknowledge-only; KIP-932 delivery state is broker-managed, no client-side mutation exists
Schema Registry Mismatch	kafka.updateSchemaCompatibility — mutate schema registry compatibility
Under-Replicated Partitions	kafka.reassignPartitions — reassign partition leadership

> **Important**
> Rejected actions are fully audited. The Monitor agent aborts and logs the rejection. No changes are made to the cluster. The history entry updates to ✗ Rejected.

4.3 Approval Status in History

When you approve or reject from the approval gate, the corresponding history entry updates automatically:

- Approve → entry shows ✓ Resolved with green badge
- Reject → entry shows ✗ Rejected with red badge and 'No lesson recorded' in the detail popup
- The entry remains in the history bar permanently for audit purposes

5. Scenarios

5.1 Common Scenarios (always visible)

Scenario	KIP	Approval?	Description
Consumer Lag Spike	KIP-848	Yes	Consumer group falling behind producer. Monitor scales consumers after approval.
KRaft Controller Failover	KRaft	No	Controller election. Monitor acknowledges and logs the new epoch.
Share Group Rebalance	KIP-932	Yes	Share group rebalance event. Monitor acknowledges after approval (no client-side mutation exists).
False-Positive Suppression	KIP-848	No	Normal rebalance noise. Monitor suppresses — demonstrates AI intelligence.

5.2 More Scenarios (expandable)

Scenario	Gate	Approval?	Description
Schema Registry Mismatch	Avro	Yes	Schema compatibility violation detected and resolved.
Broker Disk Saturation	I/O	No	Broker disk near capacity — auto-remediation triggers.
Under-Replicated Partitions	ISR	Yes	Under-replicated partitions. Reassignment requires approval.
Producer Timeout Storm	Batch	No	Producer ACK timeouts — auto-diagnosed and tuned.
Consumer Session Timeout	GC	No	Consumer heartbeat failure — auto-rebalance triggered.
Log Compaction Lag	Compact	No	Log compaction falling behind — auto-remediation.
Partition Imbalance	Suppress	No	Leader imbalance detected — suppressed as benign.

6. Lessons Learned

After every completed scenario the Monitor agent records a lesson in ops.lessons.v1. Lessons appear in two places:

- Scenario detail popup — the 📚 Learn section at the bottom shows the lesson card for that specific run
- Lessons Learned panel (right sidebar, bottom) — persistent history of all lessons across all sessions

Up to 100 lessons are stored in localStorage so they survive page refresh. Each card shows the scenario ID, action taken, truncated notes, timestamp, and an Effective / Needs Review badge. Click any card to open the Lesson Detail modal with all fields — action taken, lag before/after, adjusted threshold, effectiveness, and full notes.

7. Kafka Cluster — Confluent for Kubernetes on DigitalOcean

7.1 Cluster Statistics Panel

Click the REAL mode badge in the top bar to open the Cluster Statistics modal. It shows:

- Bootstrap Host — DigitalOcean NodePort external IP (64.227.25.232)
- Internal (K8s) — kafka.confluent.svc.cluster.local:9071 (in-cluster DNS)
- Port, SASL Mechanism, Auth User, CA Certificate status, Password status
- Cluster Status grid: MODE, BROKERS, TOPICS, CTRL EPOCH, ACLS, MTLS
- Topics table: name, partitions, current lag (red if > 0), latest offset
- Consumer Groups section with group ID, member count, and lag

7.2 Connection Details

Property	Value
Provider	Confluent for Kubernetes (CFK) 8.x, hosted on DigitalOcean Kubernetes Service (DOKS)
Kafka Version	4.2.0 — KRaft mode (no ZooKeeper)
External Bootstrap	64.227.25.232:31090 (DOKS NodePort, PLAINTEXT)
Internal Bootstrap	kafka.confluent.svc.cluster.local:9071
Brokers	3 (kafka-0, kafka-1, kafka-2)
KRaft Controllers	3 (controller-0/1/2)
Authentication	None (no-auth demo cluster)
Replication Factor	3 (all topics)

7.3 Topics

Topic	Partitions	Retention	Cleanup
ops.requests.v1	6	7 days	delete
ops.kafka.metrics.v1	6	1 day	delete
ops.incidents.v1	6	30 days	delete

ops.actions.audit.v1	12	365 days	delete
ops.lessons.v1	3	Unlimited	compact
ops.notifications.v1	3	7 days	delete
demo.payments.events	24	3 days	delete

8. Environment Variables (REAL mode)

Variable
KAFKA_MODE
KAFKA_BOOTSTRAP
KAFKA_BOOTSTRAP_SERVERS
KAFKA_SSL_ENABLED
KUBECONFIG_BASE64

9. Keyboard Shortcuts & UX Tips

Action	How
Close any modal	Press Escape or click ✕ button
Pause auto-trigger	Click ⏸ Pause in the scenario panel header
Trigger a scenario manually	Click any scenario button — works even when paused
Open approval gate	Click Review & Decide (right panel) or Review → (history bar)
Send history entry for approval	Click the history entry → Send for Approval → button
View broker details	Click REAL mode badge in top bar → Cluster Statistics
Reset all state	Click Reset in the top bar — clears canvas, history, approvals

10. Troubleshooting

Issue	Fix
Broker shows MOCK after setting KAFKA_MODE	Ensure value is lowercase 'real'. Restart with rm -rf .next && npm run dev.
Approval gate not appearing	Trigger Consumer Lag Spike or Share Group Rebalance. Wait ~6 seconds for reasoning phase.
Send for Approval button not visible	The button only appears on entries that are genuinely awaiting approval (not resolved or rejected).

History entry still shows Awaiting after decision	The status updates in local state. Hard refresh (Cmd+Shift+R) if it persists.
Root causes not showing in popup	All entries should show 🎯 Root Cause and ⚠️ Available Root Causes. Ensure latest deployment.
Scenarios firing too fast	Click ⏸️ Pause. Vercel fires every 30–40s by design. Resume when ready.
Lessons not showing in sidebar	Run at least one scenario. Lessons stored in localStorage — incognito clears them.
SSE stream disconnects	Normal on Vercel (60s timeout). Client auto-reconnects. Green Live badge in top bar.
Bootstrap connection refused	Check the DOKS cluster: kubectl get pods -n confluent. Verify KAFKA_BOOTSTRAP_SERVERS and KAFKA_BOOTSTRAP both match 64.227.25.232:31090 in Vercel env vars.
Every Kafka route fails with a TLS or connection error	Check KAFKA_SSL_ENABLED is exactly "true" or "false" in Vercel — an empty or missing value previously defaulted to TLS-on against the plaintext listener. The app now throws a clear error naming the bad value in real mode.
Escape not closing a modal	Click anywhere on the page first to ensure focus, then press Escape.